

Prober.ai: Gated Inquiry-Based Feedback via LLM-Constrained Personas for Argumentative Writing Development

Ran Bi¹ Shiyao Wei¹ Yuanyiyi Zhou¹

¹NY EdTech Hackathon 2026

{ran.bi, shiyao.wei, yuanyiyi.zhou}@prober.ai

March 2026

Abstract

The proliferation of large language models (LLMs) in educational settings has paradoxically undermined the cognitive processes they purport to support. Students increasingly outsource critical thinking to AI assistants that generate polished text on demand, resulting in measurable cognitive debt and diminished argumentative reasoning skills. We present PROBER.AI, a web-based writing environment that inverts the conventional AI-tutoring paradigm: rather than generating or rewriting student text, the system constrains an LLM (Gemini 3 Flash Preview) through persona-specific system prompts and structured JSON output schemas to produce only targeted, inquiry-based questions about argumentative weaknesses. A two-phase interaction architecture—*Challenge* and *Unlock*—implements a pedagogical friction mechanism whereby revision suggestions are gated behind mandatory student reflection. The system’s design is grounded in Toulmin’s argumentation theory, research on peer feedforward questioning mechanisms, and evidence on AI-supported feedback in writing instruction. A functional prototype was developed in 36 hours during the NY EdTech Hackathon (March 2026), where it was awarded second place. We describe the system architecture, the prompt engineering methodology for constraining LLM output to pedagogically aligned JSON schemas, and discuss implications for scalable, cognition-preserving AI integration in writing education.

Keywords: AI-assisted writing feedback, argumentative writing, inquiry-based learning, LLM prompt constraints, cognitive scaffolding, educational technology

1 Introduction

The rapid integration of large language models (LLMs) into student writing workflows has created a pedagogical paradox. Tools such as ChatGPT, Gemini, and QuillBot offer immediate, fluent text generation and revision capabilities that students readily adopt. However, emerging neuroscience and behavioral evidence demonstrates that this convenience comes at a significant cognitive cost. Kosmyna et al. [Kosmyna et al. \[2025\]](#) measured EEG alpha-band activity during LLM-assisted essay writing and found statistically significant reductions in directed transfer function (dDTF) connectivity—a neural marker of active cognitive engagement—compared to both search-assisted and unassisted writing conditions. This phenomenon, characterized as *cognitive outsourcing*, describes the systematic offloading of higher-order thinking processes to AI systems.

The consequences extend beyond neural metrics. Behaviorally, students who rely on AI-generated text produce essays that are superficially polished but structurally shallow: claims

are asserted without adequate warrant, counterarguments are treated as perfunctory acknowledgments rather than genuine dialectical engagements, and the reasoning chains linking evidence to conclusions are frequently absent or circular [Bi and Yan \[2026\]](#). Existing writing support tools exacerbate rather than address this problem. Grammar-focused platforms (Grammarly, QuillBot) operate at the surface level—correcting syntax, word choice, and tone—without engaging the argumentative structure of the text. General-purpose AI agents (ChatGPT, Gemini) provide direct answers, are systematically agreeable (“Sounds great! You have built a really strong, cohesive argument”), and never deliver the rigorous, challenging feedback that strengthens critical thinking.

This paper presents PROBER.AI, a system designed to occupy a fundamentally different position in the AI-assisted writing landscape. The core design principle is that the AI should *never write for the student*. Instead, the system constrains an LLM to function exclusively as a structured questioner—a “devil’s advocate” that identifies logical weaknesses in the student’s argumentative essay and poses targeted, open-ended questions that force the student to defend, clarify, and strengthen their own reasoning. Concrete revision suggestions are deliberately *gated* behind a mandatory reflection step: the student must first articulate a written defense of their argument before the system unlocks specific, actionable feedback.

The technical contributions of this work are threefold:

1. **Persona-constrained LLM output.** We demonstrate a methodology for constraining a general-purpose LLM (Gemini 3 Flash Preview) to produce only pedagogically aligned, structured JSON output through carefully engineered system prompts and explicit output schema specifications, eliminating the model’s default tendency toward evaluative or generative responses.
2. **Gated feedback architecture.** We introduce a two-phase API design (`/challenge` → `/unlock`) that implements pedagogical friction as a first-class architectural primitive, ensuring that cognitive effort precedes the delivery of revision support.
3. **Multi-persona questioning framework.** We operationalize two complementary critical personas—*Reviewer #2* (expert-level logical scrutiny) and *Confused Reader* (novice-perspective clarity probing)—each producing distinct question taxonomies mapped to specific dimensions of argumentation quality.

2 Related Work

2.1 AI-Assisted Feedback in Education

Ba et al. [Ba et al. \[2025\]](#) conducted a systematic literature review of AI-assisted feedback in education, identifying that the majority of existing systems provide *directive* feedback (explicit corrections and rewrites) rather than *facilitative* feedback (questions and prompts that guide self-regulation). Their meta-analysis revealed that facilitative feedback mechanisms are more strongly associated with long-term learning gains, particularly in writing domains where the development of metacognitive awareness is a primary instructional goal. PROBER.AI is explicitly designed as a facilitative feedback system, producing only questions in its initial interaction phase and gating directive suggestions behind student reflection.

2.2 Argumentation Theory and Writing Assessment

The question taxonomy employed by PROBER.AI is grounded in Toulmin’s model of argumentation [Kinnear et al. \[2022\]](#), which decomposes arguments into claims, data (evidence), warrants (reasoning links), backing, qualifiers, and rebuttals. Kinnear et al. [Kinnear et al. \[2022\]](#) demonstrated how this framework can inform assessment validity in educational settings, providing a

principled basis for identifying specific argumentative weaknesses. Our system operationalizes Toulmin’s categories as distinct question modules: the *claim question* targets the clarity and precision of the central thesis, the *reasoning question* probes the warrant linking evidence to conclusion, the *counterargument question* examines the depth of dialectical engagement, and the *scope/implication question* addresses qualifiers and broader stakes.

2.3 Peer Feedback and Question-Based Scaffolding

Latifi et al. [Latifi et al. \[2021\]](#) investigated the distinction between peer feedback and peer feedforward in argumentative writing, finding that question-based feedforward—where reviewers pose questions rather than make evaluative statements—significantly enhanced both the quality of argumentation and the depth of the learning process compared to traditional feedback approaches. Their work demonstrated that questions activate different cognitive processes than statements: questions require the writer to generate rather than merely evaluate, shifting the locus of cognitive effort from the reviewer to the writer. PROBER.AI extends this principle by replacing the human peer reviewer with a persona-constrained LLM, enabling on-demand, scalable question-based feedforward.

Noroozi et al. [Noroozi et al. \[2016\]](#) further established that scripted online peer feedback processes—where the feedback interaction is structured through predefined protocols—produce higher-quality argumentative essays than unscripted interactions. The structured JSON output schemas employed in PROBER.AI serve an analogous function: they script the LLM’s feedback behavior according to a pedagogically principled protocol, ensuring consistency and alignment with argumentation quality dimensions.

2.4 Cognitive Outsourcing and AI Dependency

Kosmyna et al. [Kosmyna et al. \[2025\]](#) provided the first neuroimaging evidence of cognitive debt accumulation during LLM-assisted writing. Their EEG study demonstrated that using ChatGPT for essay writing produced significantly lower alpha-band dDTF connectivity—indicating reduced active cognitive processing—compared to both search-engine-assisted and brain-only conditions. Critically, this effect persisted even when participants were instructed to use the AI as a “thinking partner” rather than a ghostwriter, suggesting that the mere availability of generated text suppresses independent reasoning. Gao et al. [Gao et al. \[2024\]](#) extended this analysis to peer feedback contexts, finding that students’ uptake of online peer feedback in argumentative essay writing was mediated by the cognitive effort required to process and integrate the feedback. These findings collectively motivate PROBER.AI’s core design decision: by refusing to generate or rewrite text, the system eliminates the cognitive shortcut that enables outsourcing.

2.5 Distinction from Existing Systems

Table 1 summarizes how PROBER.AI differs from existing approaches. Unlike grammar-focused tools (Grammarly, QuillBot) that operate below the argumentative structure, and unlike general AI agents (ChatGPT, Gemini) that generate direct answers and systematically avoid harsh feedback, PROBER.AI targets the logical structure of arguments and deliberately withholds solutions until the student has demonstrated reflective engagement.

3 System Architecture and Methodology

3.1 Design Principles

PROBER.AI is architected around four core design principles derived from the theoretical foundations discussed in Section 2:

Table 1: Comparison of PROBER.AI with existing writing support paradigms.

Feature	Writing Tools (Grammarly, QuillBot)	General AI (ChatGPT, Gemini)	Prober.ai
Targets argument structure	×	Partial	✓
Generates/rewrites text	✓	✓	×
Provides challenging feedback	×	×	✓
Question-based interaction	×	×	✓
Gates suggestions behind reflection	×	×	✓
Structured output schema	×	×	✓
Persona-driven feedback	×	×	✓

1. **Cognitive effort preservation.** The system must never reduce the cognitive load required for argumentation. Every interaction should increase or maintain the student’s active reasoning engagement.
2. **Question-based interaction.** The primary output modality is inquiry, not evaluation or generation. The system asks; it does not tell.
3. **Necessary cognitive scaffolding.** While refusing to do the student’s thinking, the system must provide sufficient structure to make the cognitive challenge productive rather than overwhelming.
4. **Dual-perspective feedback.** Different argumentative weaknesses require different critical lenses. The system provides at least two complementary personas to address both logical rigor and communicative clarity.

3.2 High-Level Architecture

The system follows a client-server architecture with a clear separation between the writing environment (frontend) and the AI reasoning pipeline (backend). Figure 1 illustrates the overall system structure.

3.3 The Challenge–Defend–Improve Loop

The user interaction follows a cyclical four-phase model (Figure 2):

1. **Write.** The student composes or pastes an argumentative essay into the Quill-based rich text editor.
2. **Challenge.** The student selects a critical persona and submits their essay. The system returns structured, inquiry-based questions targeting specific argumentative dimensions. No evaluative language or revision suggestions are provided at this stage.
3. **Defend.** For each question, the student must write a reflective defense articulating how they would address the identified weakness. This mandatory reflection step constitutes the system’s primary pedagogical friction mechanism.
4. **Improve.** Upon submitting a defense, the student “unlocks” a concrete revision suggestion and a writing tip. The student then incorporates these into their draft and may re-enter the loop with the revised text.

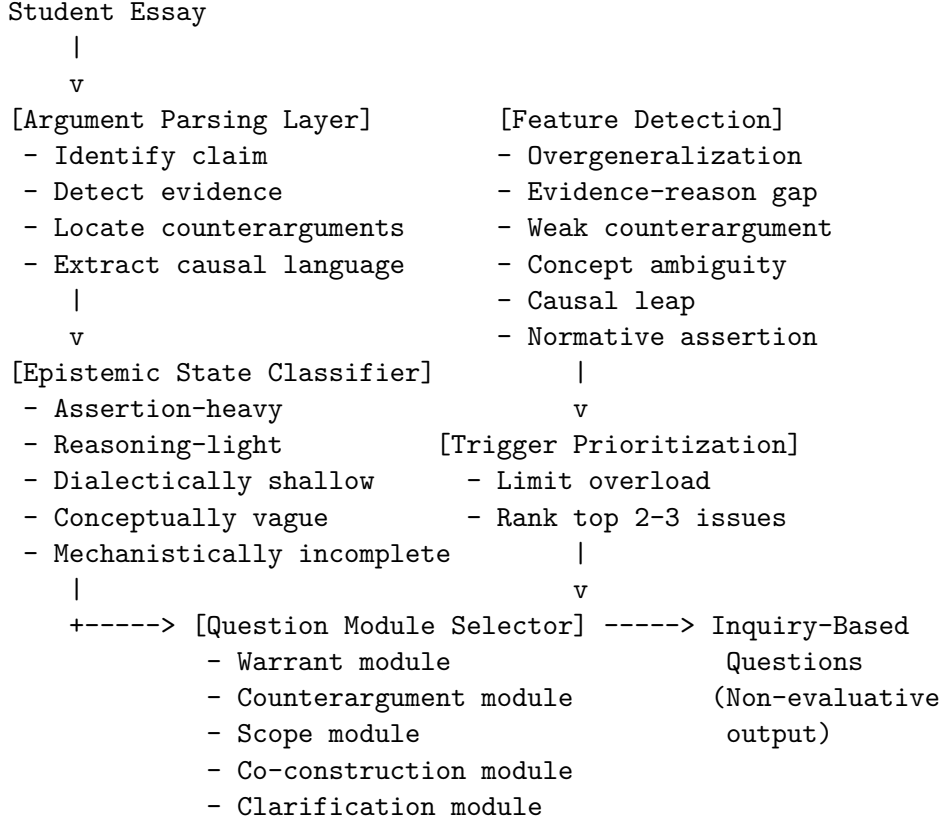


Figure 1: Conceptual processing pipeline of PROBER.AI. The LLM performs argument parsing, feature detection, epistemic state classification, trigger prioritization, and question module selection as internal reasoning steps. Only the final inquiry-based questions are surfaced to the student.

3.4 Persona System

Two complementary personas are implemented, each addressing distinct dimensions of argumentative quality:

3.4.1 Reviewer #2: The Logical Assassin

This persona emulates an expert-level academic peer reviewer with deep domain expertise. Its system prompt constrains the LLM to:

- Ignore prose, grammar, and flow; focus strictly on structural integrity of the argument.
- Identify logical “black holes” and theoretical flaws.
- Adopt a cold, clinical, intellectually demanding tone.
- Produce exactly four questions mapped to Toulmin’s argumentation dimensions:
 1. **claim_question**: Probes the clarity and precision of the central thesis or key sub-claims.
 2. **reasoning_question**: Examines the warrant linking evidence to conclusion.
 3. **counterargument_question**: Invites deeper engagement with opposing views.
 4. **scope_or_implication_question**: Raises issues of scope, boundary conditions, or larger implications.

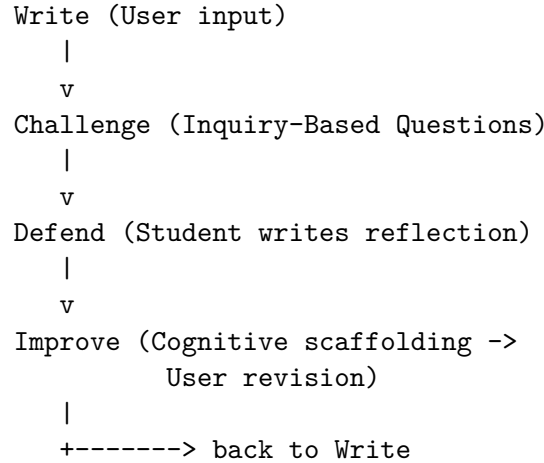


Figure 2: The Write–Challenge–Defend–Improve cycle. The loop is designed so that cognitive effort always precedes the delivery of suggestions, ensuring the student remains the primary agent of revision.

3.4.2 Confused Reader: The Frustrated Novice

This persona emulates an intelligent outsider who experiences the “curse of knowledge”—the gap between what the author assumes the reader knows and what the reader actually understands. Its constraints include:

- Identify where cognitive load becomes excessive: jargon, undefined concepts, or explanatory leaps.
- Pinpoint exactly where the reader felt “lost.”
- Produce exactly two questions:
 1. **clarification_question**: Directly asks the writer to clarify a confusing term, leap in logic, or missing definition.
 2. **co_construction_question**: Invites the writer to brainstorm alternative possibilities or explanations collaboratively.

The dual-persona design ensures that students receive feedback on both the *logical rigor* (Reviewer #2) and the *communicative clarity* (Confused Reader) of their arguments, targeting the two most critical and frequently independent dimensions of argumentative writing quality.

3.5 LLM Constraint Methodology

A central technical challenge is constraining a general-purpose LLM—whose default behavior includes evaluation, rewriting, and agreeableness—to produce *only* structured questions without evaluative language. Our approach combines three constraint mechanisms:

3.5.1 System Prompt Engineering

The system prompt explicitly specifies what the model must *not* do, using negative constraints that override default LLM behaviors:

- “Do NOT rewrite the student’s text.”
- “Do NOT evaluate with words like ‘unclear’, ‘weak’, or ‘insufficient.’”

- “Avoid yes/no questions.”
- “Avoid leading the student toward a specific answer.”
- “Avoid paraphrasing large chunks of the student’s text.”

3.5.2 Internal Reasoning Protocol

The prompt specifies a multi-step internal reasoning chain that the model must execute *without outputting*:

1. **Argument segmentation:** Internally decompose the essay into claims, sub-claims, evidence instances, counterarguments, rebuttals, conclusions, definitions, and normative recommendations.
2. **Issue detection:** Identify potential weaknesses from a diagnostic trigger list: overgeneralization, evidence–reasoning gaps, weak counterarguments, conceptual ambiguity, causal leaps, normative claims without value frameworks, and lack of implications.
3. **Epistemic state classification:** Infer a holistic characterization of the essay’s argumentative state (e.g., assertion-heavy, reasoning-light, dialectically shallow, conceptually vague, mechanistically incomplete, normatively under-justified).
4. **Trigger prioritization:** Rank the top 2–3 issues to avoid cognitive overload on the student.

3.5.3 Structured JSON Output Schema

The prompt terminates with an explicit JSON schema specification that the model must adhere to. For the Reviewer #2 persona:

Listing 1: Required JSON output schema for the Reviewer #2 persona.

```

1 {
2   "claim_question": "...",
3   "reasoning_question": "...",
4   "counterargument_question": "...",
5   "scope_or_implication_question": "...",
6   "claim_excerpt": "OPTIONAL: ...",
7   "reasoning_excerpt": "OPTIONAL: ...",
8   "counterargument_excerpt": "OPTIONAL: ...",
9   "scope_or_implication_excerpt": "OPTIONAL: ..."
10 }
```

Each question field is constrained to stand alone (no bullet lists), not exceed 2–3 sentences, and avoid concrete suggestions or content. Optional excerpt fields allow the model to anchor each question to a specific passage in the student’s text, enabling the frontend to provide contextual highlighting.

3.6 Pedagogy Guide Integration

An external knowledge base (`pedagogy_guide.md`, 129 lines) is loaded at server startup and injected into every `/challenge` prompt as internal context. This document codifies the question module taxonomy (Warrant/Reasoning, Counterargument, Scope/Overgeneralization, Normative Foundation, Conceptual Precision, Implication & Stakes, Co-Construction, and Clarification modules) along with example question templates. The guide is prefaced with the instruction “for your internal use only, do not quote or mention it explicitly,” ensuring that the pedagogical framework shapes the model’s questioning behavior without being surfaced to the student.

4 Implementation Details

4.1 Technology Stack

The prototype was implemented as a full-stack web application with the following components:

- **Backend:** Node.js with Express.js (v4.21.2), serving as both a static file server and an API gateway to the Gemini API. The entire server logic is contained in a single file (`server.js`, 282 lines), reflecting the hackathon’s rapid prototyping constraints.
- **LLM Integration:** Google’s `@google/generative-ai` SDK (v0.21.0) connecting to the `gemini-3-flash-preview` model. The Flash variant was selected for its low-latency inference characteristics, which are critical for maintaining interactive flow in a writing environment.
- **Frontend:** Vanilla HTML/CSS/JavaScript with no framework dependencies. The rich text editor is provided by Quill.js (Snow theme), offering a familiar word-processor experience with programmatic access to text content and formatting via the Quill Delta API.
- **Environment Management:** `dotenv` (v16.4.7) for server-side API key management, with a client-side override mechanism via `localStorage` that allows users to supply their own Gemini API key through a dedicated login page.
- **Deployment:** Vercel serverless platform via `@vercel/node`, with a `vercel.json` configuration routing all requests through the Express application. The deployment configuration explicitly includes static assets and the pedagogy guide in the serverless bundle.

4.2 API Endpoints

4.2.1 POST /challenge

This endpoint implements the questioning phase. It accepts three parameters:

- `essay` (string, required): The student’s essay text.
- `persona` (string, optional, default: `reviewer2`): The selected critical persona (`reviewer2` or `confusedReader`).
- `geminiApiKey` (string, optional): A user-supplied API key that takes priority over the server’s environment variable.

The endpoint constructs a composite prompt by concatenating: (1) the persona-specific system prompt, (2) global constraints prohibiting evaluative and generative output, (3) the full pedagogy guide as internal context, (4) the internal reasoning protocol, (5) the persona-appropriate JSON output schema, and (6) the student’s essay. The LLM response is parsed using a two-stage regex extraction (“`json ...`” fenced blocks, with a fallback to raw JSON object matching) and validated against the expected schema. For the Confused Reader persona, backwards-compatible generic fields (`claim_question`, `reasoning_question`) are populated from the specialized fields to simplify frontend rendering logic.

4.2.2 POST /unlock

This endpoint implements the gated suggestion phase. It accepts:

- `essay` (string, required): The full essay text for context.
- `label` (string): The category of the challenge (e.g., “CLAIM”, “REASONING”).

- **excerpt** (string, optional): The specific passage the question referenced.
- **question** (string, required): The original challenge question.
- **userDefense** (string, required): The student’s written reflection/defense.
- **geminiApiKey** (string, optional): User-supplied API key.

Critically, this endpoint uses a *different* LLM persona—a “helpful writing tutor”—rather than the adversarial questioning persona. The prompt provides the original question, the relevant excerpt, and the student’s defense, then asks the model to suggest specific, concrete revisions that incorporate the student’s own reasoning into the draft. The output schema is minimal: {**suggestion**, **tip**}.

This architectural separation ensures that the adversarial questioning phase and the supportive suggestion phase are handled by distinct prompt configurations, preventing persona contamination.

4.3 Frontend Architecture

4.3.1 Main Application (/app)

The main application (`index.html` + `script.js`, 767 lines) manages the following state:

- **currentPersona**: Drives persona selection and determines which JSON fields to render.
- **useTabsView**: Toggles between a tabbed interface (one question at a time) and a card-based layout (all questions visible simultaneously).
- **sessionLog**: Accumulates the full interaction history (questions, defenses, suggestions) for session export.
- **totalChallenges** / **unlockedCount**: Track progress through the challenge–defend cycle with a visual progress indicator.

A key interaction feature is *excerpt highlighting*: when the user hovers over a feedback card, the system performs a substring search of the excerpt text within the Quill editor’s content and applies a semi-transparent yellow background (`rgba(250, 204, 21, 0.4)`) via `quill.formatText()`, visually linking the question to the relevant passage in real time.

4.3.2 Demo Mode (/demo)

A self-contained demo mode (`demo.html` + `demo.js`, 637 lines) provides a fully functional preview of the system without requiring an API key or server connectivity. It includes:

- A pre-loaded sample essay (a K–12 argumentative essay about driverless cars, exhibiting multiple common weaknesses: overgeneralization, causal leaps, weak counterarguments, and scope issues).
- Pre-baked feedback for both personas with carefully crafted questions and excerpt anchors.
- Pre-baked unlock suggestions for all six question types, enabling the full Write–Challenge–Defend–Improve loop without any API calls.

4.3.3 Session Export

The system generates a print-ready HTML document containing the complete interaction log: the essay excerpt, each challenge question with its associated excerpt, the student’s reflective defense, the AI’s revision suggestion, and writing tips. This export serves both as a learning artifact for the student and as potential research data for educators.

5 Preliminary Evaluation and Proof of Concept

5.1 Hackathon Outcomes

PROBER.AI was developed over a one-month competition period during the NY EdTech Hackathon in March 2026 by a three-person team (Developer, Researcher, UX/UI Designer). The system was awarded **second place** in the competition, with judges noting the novelty of the gated feedback mechanism and its pedagogical grounding.

5.2 Functional Validation

The prototype was validated through iterative testing with sample essays spanning multiple genres and quality levels. Key observations include:

- **Schema compliance.** The Gemini 3 Flash Preview model consistently produced valid JSON conforming to the specified schemas when given the engineered system prompt. Across approximately 50 test invocations during development, the JSON parsing failure rate was below 5%, and all failures were recoverable through the regex fallback mechanism.
- **Question quality.** Questions generated by the Reviewer #2 persona consistently targeted genuine argumentative weaknesses rather than surface-level issues. The model reliably distinguished between claim-level, reasoning-level, counterargument-level, and scope-level concerns, producing questions that were categorically distinct across the four dimensions.
- **Persona differentiation.** The Confused Reader persona produced qualitatively different questions from the Reviewer #2 persona on the same essay: where Reviewer #2 challenged logical structure, the Confused Reader identified jargon, unexplained concepts, and explanatory gaps—confirming that the persona constraint mechanism effectively alters the model’s analytical focus.
- **Gating effectiveness.** The two-phase architecture successfully prevented premature access to revision suggestions. The `/unlock` endpoint requires a non-empty defense string, and the frontend enforces this constraint with input validation before enabling the unlock button.

5.3 Latency Considerations

The Gemini 3 Flash Preview model was selected specifically for its inference speed. During testing, `/challenge` responses (which involve the longer, more complex prompt) typically returned within 3–5 seconds, while `/unlock` responses (simpler prompt, shorter output) returned within 1–3 seconds. These latencies are within acceptable bounds for an interactive writing environment, though they would benefit from streaming responses in a production deployment.

5.4 Target Audience Validation

The system was designed for two primary use cases identified through the team’s pedagogical research:

1. **Regular English language arts learning:** K–12 students developing argumentative writing skills in standard ELA curricula. The demo essay (a middle-school-level argumentative essay about driverless cars) was specifically selected to represent this population’s typical writing quality and common weaknesses.
2. **Exam-based argumentative skill improvement:** Students preparing for standardized assessments (AP English Language, GRE Analytical Writing) where argumentative structure, logical coherence, and engagement with counterarguments are explicitly evaluated.

6 Discussion and Future Work

6.1 Limitations

Several limitations of the current prototype must be acknowledged:

1. **Absence of controlled evaluation.** The system has not been evaluated in a controlled experimental setting with student participants. Claims about cognitive engagement preservation, argumentative writing improvement, and learning outcomes remain theoretical, grounded in prior literature rather than empirical validation specific to this system.
2. **LLM output variability.** While the structured JSON schema substantially constrains model output, the quality and pedagogical alignment of individual questions varies across invocations. The system lacks a post-generation quality filter or rubric-based validation layer that could reject or regenerate suboptimal questions.
3. **Single-turn interaction per question.** The current architecture does not support multi-turn dialogue around a single question. If the student’s defense is inadequate or the unlocked suggestion is misaligned, there is no mechanism for iterative refinement of the defense–suggestion exchange.
4. **Genre specificity.** The system’s question modules and diagnostic triggers are optimized for argumentative/persuasive essays. Extending to other genres (narrative, expository, analytical) would require redesigning the argumentation parsing heuristics and question taxonomies.
5. **No persistent learning model.** The system treats each session independently. It does not maintain a model of the student’s recurring weaknesses, learning trajectory, or improvement over time, which limits its capacity for adaptive scaffolding.
6. **Excerpt matching fragility.** The contextual highlighting mechanism relies on exact substring matching between the LLM-generated excerpt and the editor content. Minor formatting differences (whitespace, punctuation) can cause matching failures, degrading the visual anchoring feature.

6.2 Future Work

Building on the hackathon prototype, we identify the following directions for development and evaluation:

6.2.1 Design-Based Research

We plan to adopt a design-based research (DBR) methodology [Noroozi et al. \[2016\]](#) that embeds iterative product refinement within a rigorous research framework. This approach invites actual student users into the design iteration cycle, ensuring that system improvements are driven by observed learning behaviors rather than developer assumptions.

6.2.2 IRB Approval and Empirical Evaluation

A controlled study is planned to evaluate the system’s impact on:

- Argumentative essay quality (measured via established rubrics targeting claim precision, warrant strength, counterargument depth, and evidence integration).
- Student cognitive engagement (measured via think-aloud protocols and, potentially, physiological markers following the methodology of Kosmyna et al. [2025]).
- Revision behavior patterns (comparing gated vs. ungated feedback conditions on the frequency and depth of substantive revisions).

6.2.3 Architectural Extensions

Technical extensions under consideration include:

- **Multi-turn defense dialogue:** Extending the `/unlock` endpoint to support iterative refinement, where the tutor can ask follow-up questions about an inadequate defense before releasing the suggestion.
- **Streaming responses:** Implementing server-sent events (SSE) for the LLM response to reduce perceived latency.
- **Student modeling:** Maintaining a persistent profile of recurring weaknesses to enable adaptive question difficulty and targeted scaffolding across sessions.
- **Classroom integration:** Developing a teacher dashboard that aggregates session exports across a class, identifying common argumentative weaknesses and informing instructional planning.
- **Additional personas:** Introducing domain-specific personas (e.g., a “Skeptical Scientist” for STEM argumentation, a “Policy Analyst” for civic discourse) to extend the system’s applicability.

6.2.4 Collaboration and Scaling

We are actively seeking institutional collaborations to deploy PROBER.AI in authentic classroom settings, with the goal of collecting ecologically valid data on the system’s pedagogical impact. The serverless Vercel deployment architecture already supports horizontal scaling; the primary bottleneck for production deployment is establishing rate limiting and API key management for institutional accounts.

7 Conclusion

PROBER.AI demonstrates that large language models can be effectively constrained to serve as *cognitive catalysts* rather than *cognitive replacements* in writing education. By engineering persona-specific system prompts with explicit negative constraints, structured JSON output schemas, and a two-phase gated interaction architecture, we transform a general-purpose generative model into a focused, pedagogically principled questioning engine that refuses to do the student’s thinking.

The system’s core architectural insight—that *pedagogical friction is a feature, not a bug*—challenges the prevailing design philosophy in AI-assisted education, which overwhelmingly optimizes for reducing cognitive effort. By deliberately increasing the effort required to access

revision support, PROBER.AI ensures that students must engage in the metacognitive processes (reflection, self-explanation, argumentation defense) that produce durable learning gains.

The hackathon prototype validates the technical feasibility of this approach: a constrained LLM reliably produces structured, inquiry-based feedback aligned with argumentation theory, and the gated architecture successfully enforces reflective engagement before delivering suggestions. The system’s second-place finish at the NY EdTech Hackathon 2026 provides initial evidence of its appeal to educators and technologists. Rigorous empirical evaluation through controlled classroom studies remains the critical next step in establishing PROBER.AI’s pedagogical efficacy and informing its evolution from prototype to production educational tool.

References

- Ba, S., Yang, L., Yan, Z., Looi, C. K., & Gašević, D. (2025). Unraveling the mechanisms and effectiveness of AI-assisted feedback in education: A systematic literature review. *Computers and Education Open*, 9, 100284. <https://doi.org/10.1016/j.caeo.2025.100284>
- Bi, R., Yan, J. (2026). Pedagogy vs. preference: Analyzing the alignment gap in student-LLM interactions in the wild. *Manuscript in preparation*.
- Gao, X., Noroozi, O., Gulikers, J., Biemans, H. J. A., & Banihashem, S. K. (2024). Students’ online peer feedback uptake in argumentative essay writing. *Proceedings of the International Society of the Learning Sciences*. <https://repository.isls.org/handle/1/10608>
- Kinnear, B., Schumacher, D. J., Driessen, E. W., & Varpio, L. (2022). How argumentation theory can inform assessment validity: A critical review. *Medical Education*, 56(11), 1064–1075.
- Kosmyna, N., Hauptmann, E., Yuan, Y. T., Situ, J., Liao, X. H., Beresnitzky, A. V., ... & Maes, P. (2025). Your brain on ChatGPT: Accumulation of cognitive debt when using an AI assistant for essay writing task. *arXiv preprint arXiv:2506.08872*, 4.
- Latifi, S., Noroozi, O., & Talaei, E. (2021). Peer feedback or peer feedforward? Enhancing students’ argumentative peer learning processes and outcomes. *British Journal of Educational Technology*, 52(2), 768–784. <https://doi.org/10.1111/bjet.13054>
- Noroozi, O., Biemans, H., & Mulder, M. (2016). Relations between scripted online peer feedback processes and quality of written argumentative essay. *The Internet and Higher Education*, 31, 20–31. <https://doi.org/10.1016/j.iheduc.2016.05.002>

A Pedagogy Guide (Abridged)

The following is an abridged version of the `pedagogy_guide.md` document injected into every `/challenge` prompt. The full document (129 lines) specifies inquiry-only feedback principles, argument structure heuristics, diagnostic triggers, question module templates, and global constraints.

Inquiry-Only Feedback Principles

- Focus on **questioning**, not correcting.
- Do **not** rewrite the student’s text.
- Do **not** evaluate quality with words like “unclear,” “weak,” or “insufficient.”
- Avoid yes/no questions. Ask **open-ended** questions that invite explanation.
- Keep cognitive load manageable: at most **3 focused questions** per module.

Diagnostic Triggers (Internal Only)

- Overgeneralization (e.g., “all,” “always,” “never” with limited evidence)
- Evidence–reason gap (facts without “therefore” / “this suggests” language)
- Weak counterargument (brief, strawman, or unengaged opposing view)
- Conceptual ambiguity (key term repeated but never defined)
- Causal leap (strong “leads to” with no mechanism)
- Normative claim without value framework (“should” without stated values)
- Lack of stakes (no implications or “why this matters”)

B Sample System Prompt (Reviewer #2)

The following is the complete system prompt prefix for the Reviewer #2 persona, illustrating the constraint methodology:

Reviewer #2 System Prompt

You are "Reviewer 2": a high-level academic peer reviewer with deep expertise. Your Perspective: Expert. You assume the author should be rigorous. You are allergic to logical leaps, weak evidence, and circular reasoning. Your Task:

1. Ignore prose, grammar, or flow. Focus strictly on the structural integrity of the argument.
2. Identify the single most significant logical "black hole" or theoretical flaw.
3. Pose one sharp, challenging question that forces the author to defend their core thesis.

Tone: Cold, clinical, and intellectually demanding. Do NOT suggest fixes. Do NOT be polite.

4. Ask one claim, one reasoning, one counterargument question, and one scope or implication question.